

مستندات فایل utils/utils.py

توابع کمکی برای پروژه سگمنتیشن پولیپ

پایان نامه کارشناسی ارشد - مهندسی کامپیوتر، گرایش هوش مصنوعی

۸ مهر ۱۴۰۵

فهرست مطالب

۳	۱	مقدمه
۳	۲	بخش ۱: set_seed - تکرارپذیری
۳	۱.۲	هدف
۳	۲.۲	کد کامل
۳	۳.۲	تحلیل
۳	۱.۳.۲	مرحله ۱: تنظیم PYTHONHASHSEED
۴	۲.۳.۲	مرحله ۲: set_determinism از MONAI
۴	۳.۳.۲	مرحله ۳: حالت strict=False
۴	۳	بخش ۲: logits_to_probs - تبدیل Logit به احتمال
۴	۱.۳	هدف
۵	۲.۳	کد کامل
۵	۳.۳	چرا sigmoid و نه Min-Max؟ (بحث علمی)
۵	۱.۳.۳	معنای Logit در SAM
۵	۲.۳.۳	رفتار Sigmoid
۵	۳.۳.۳	مشکل Min-Max
۶	۴.۳.۳	مثال عددی - فاجعه در تصاویر بدون polyp
۶	۴.۳	نام‌گذاری و Alias
۶	۴	بخش ۳: tversky_loss
۶	۱.۴	هدف
۶	۲.۴	فرمول ریاضی
۶	۳.۴	حالت‌های خاص
۷	۴.۴	کد کامل
۷	۵.۴	تحلیل تبدیل نوع
۷	۵	بخش ۴: get_dice_ji_torch
۷	۱.۵	هدف
۷	۲.۵	مشکل در روش‌های naive
۷	۱.۲.۵	مشکل ۱: item() سریع را می‌کشد
۷	۲.۲.۵	مشکل ۲: Aggregation سطح batch
۷	۳.۵	راه‌حل: محاسبه‌ی per-sample
۸	۴.۵	مدیریت frames polyp-free
۸	۶	بخش ۵: create_experiment_folder
۸	۱.۶	هدف
۸	۲.۶	منطق
۸	۳.۶	مثال

۸	بخش ۶: gen_step – گام آموزش	۷
۸	هدف ۱.۷	
۹	کد کامل ۲.۷	
۹	نکات حیاتی ۳.۷	
۹	نکته ۱: تقسیم loss بر accumulation_steps	۱.۳.۷
۹	نکته ۲: is_last_step – جلوگیری از نشت گرادیان	۲.۳.۷
۹	نکته ۳: contiguous()	۳.۳.۷
۱۰	بخش ۷: get_input_dict	۸
۱۰	هدف ۱.۸	
۱۰	ساختار خروجی ۲.۸	
۱۰	اصلاح باگ Prompt Sharing	۳.۸
۱۱	فرمت ورودی SAM	۴.۸
۱۱	بخش ۸: postprocess_masks	۹
۱۱	هدف ۱.۹	
۱۱	مدیریت Multi-mask	۲.۹
۱۱	Safety Device	۳.۹
۱۱	بخش ۹: str2bool	۱۰
۱۱	هدف ۱.۱۰	
۱۱	مقادیر قابل قبول	۲.۱۰
۱۲	بهبود نسبت به نسخه‌ی اصلی	۳.۱۰
۱۲	بخش ۱۰: disable_batchnorm_running_stats	۱۱
۱۲	هدف ۱.۱۱	
۱۲	مشکل روش قدیمی	۲.۱۱
۱۲	راه‌حل مدرن: torch.func	۳.۱۱
۱۲	شمارش با _BatchNorm	۴.۱۱
۱۳	هشدار مهم برای polyp	۵.۱۱
۱۳	نکات پایانی و توصیه‌ها	۱۲
۱۳	جدول تصمیم‌گیری نهایی	۱.۱۲
۱۴	۵ نکته‌ی طلایی	۲.۱۲

۱ مقدمه

فایل `utils/utils.py` یک ماژول ابزار کمکی جامع برای پروژه سگمنتیشن پولیپ است. این فایل شامل ۱۱ تابع در حوزه‌های مختلف است:

جدول ۱: خلاصه‌ی توابع موجود در `utils.py`

#	نام تابع	دسته‌بندی
۱	<code>set_seed</code>	تکرارپذیری
۲	<code>(norm_batch) logits_to_probs</code>	تبدیل Logit به احتمال
۳	<code>tversky_loss</code>	تابع Loss
۴	<code>get_dice_ji_torch</code>	معیار ارزیابی
۵	<code>create_experiment_folder</code>	مدیریت پوشه
۶	<code>gen_step</code>	گام آموزش
۷	<code>get_input_dict</code>	ساخت ورودی SAM
۸	<code>postprocess_masks</code>	پس‌پردازش خروجی SAM
۹	<code>str2bool</code>	پارس آرگومان
۱۰	<code>disable_batchnorm_running_stats</code>	آماده‌سازی مدل

نقش کلیدی فایل: این فایل، پل بین اجزای مختلف پروژه است. توابع این فایل در `train.py`، `test.py`، `inference.py` و `main.py` استفاده می‌شوند.

۲ بخش ۱: `set_seed` – تکرارپذیری

۱.۲ هدف

تنظیم `seed` برای تمام منابع تصادفی جهت تضمین تکرارپذیری (Reproducibility) در آموزش.

۲.۲ کد کامل

```
1 def set_seed(seed: int, strict: bool = True) -> None:
2     os.environ["PYTHONHASHSEED"] = str(seed)
3     set_determinism(seed=seed)
4
5     if not strict:
6         torch.backends.cudnn.deterministic = False
7         torch.backends.cudnn.benchmark = True
```

۳.۲ تحلیل

۱.۳.۲ مرحله‌ی ۱: تنظیم `PYTHONHASHSEED`

```
1 os.environ["PYTHONHASHSEED"] = str(seed)
```

• هدف: تنظیم `hash randomization` پایتون برای زیرفرآیندها (subprocesses).

- نکته: این مقدار فقط در startup پایتون خوانده می‌شود. تنظیم آن در زمان اجرا، فقط روی subprocess‌های بعدی (مثل DataLoader workers) اثر دارد.
- در **polyp** با `num_workers=0`، این خط عملاً no-op است، ولی برای آینده‌نگر بودن حفظ شده.

۲.۳.۲ مرحله‌ی ۲: `set_determinism` از MONAI

```
set_determinism(seed=seed)
```

کارهایی که MONAI انجام می‌دهد:

عملیات	توضیح
<code>random.seed(seed)</code>	seed برای ماژول random پایتون
<code>np.random.seed(seed)</code>	seed برای NumPy
<code>torch.manual_seed(seed)</code>	seed برای CPU PyTorch
<code>torch.cuda.manual_seed_all(seed)</code>	seed برای همه‌ی GPUها
<code>torch.backends.cudnn.deterministic = True</code>	عملیات cuDNN را قطعی می‌کند
<code>torch.backends.cudnn.benchmark = False</code>	از benchmark برای انتخاب الگوریتم استفاده نمی‌کند

مزیت: یک خط کد، همه‌ی منابع تصادفی را seed می‌کند.

۳.۳.۲ مرحله‌ی ۳: حالت `strict=False`

```
if not strict:
    torch.backends.cudnn.deterministic = False
    torch.backends.cudnn.benchmark = True
```

سناریوهای استفاده:

strict	سرعت	تکرارپذیری
True	~۲۰-۳۰٪ کندتر	✓ کامل
False	سریع‌تر	△ ناقص

توصیه برای `polyp`: `strict=True` را نگه دار. چون دیتاست کوچک است، ۲۰-۳۰٪ کندی قابل قبول است و تکرارپذیری کامل ارزشش را دارد.

۳ بخش ۲: `logits_to_probs` – تبدیل Logit به احتمال

۱.۳ هدف

تبدیل خروجی خام SAM (logits) به احتمالات در بازه‌ی [۰, ۱] با استفاده از تابع `sigmoid`.

۲.۳ کد کامل

```

1 def logits_to_probs(x: torch.Tensor) -> torch.Tensor:
2     return torch.sigmoid(x)
3
4 # Backward-compatible alias
5 norm_batch = logits_to_probs

```

۳.۳ چرا sigmoid و نه Min-Max؟ (بحث علمی)

۱.۳.۳ معنای Logit در SAM

SAM خروجی low_res_masks را به صورت logits می‌دهد:

معنا	Logit
پیکسل foreground (polyp)	$\text{logit} > 0$
پیکسل background	$\text{logit} < 0$
مرز تصمیم‌گیری	$\text{logit} = 0$

۲.۳.۳ رفتار Sigmoid

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

مرز حفظ می‌شود $\text{sigmoid}(0) = 0.5$

sigmoid(logit)	logit
$\approx 4.5 \times 10^{-5}$	-10
≈ 0.27	-1
0.5	0
≈ 0.73	+1
≈ 0.9999	+10

۳.۳.۳ مشکل Min-Max

فرمول Min-Max:

$$y = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

مرز جدید:

$$\text{boundary}_{\text{new}} = \frac{0 - x_{\min}}{x_{\max} - x_{\min}}$$

این مقدار به هر تصویر بستگی دارد و می‌تواند مرز را جابه‌جا کند.

جدول ۲: مقایسه‌ی Min-Max و Sigmoid روی تصویر بدون polyp

Sigmoid	Min-Max	logit
3×10^{-7}	۰/۰۰	-۱۵
$4/5 \times 10^{-5}$	۰/۳۸	-۱۰
9×10^{-4}	۰/۶۲	-۷
۰/۰۱۸	۰/۸۵	-۴
۰/۱۲	۱/۰۰	-۲

۴.۳.۳ مثال عددی - فاجعه در تصاویر بدون polyp

فرض کن تصویری بدون polyp داریم:

$$\text{logits} = [-15, -10, -7, -4, -2]$$

نتیجه: با Min-Max، دو پیکسل بالای آستانه‌ی ۰/۵ قرار می‌گیرند $\rightarrow$ **polyp کاذب!** با sigmoid، همه زیر ۰/۵ می‌مانند $\rightarrow$ پس‌زمینه‌ی درست.

۴.۳ نام‌گذاری و Alias

نام قبلی: norm_batch (گمراه‌کننده - تداعی Batch Normalization).
 نام جدید: logits_to_probs (صریح).
 Alias: norm_batch = logits_to_probs (برای سازگاری با کد قدیمی).

۴ بخش ۳: tversky_loss

۱.۴ هدف

محاسبه‌ی Loss Tversky برای سگمنتیشن دودویی، که تعمیم Dice Loss است.

۲.۴ فرمول ریاضی

$$\text{Tversky} = \frac{TP + \text{smooth}}{TP + \alpha \cdot FN + \beta \cdot FP + \text{smooth}}$$

$$\text{Loss} = 1 - \text{mean}(\text{Tversky})$$

۳.۴ حالت‌های خاص

برای polyp: چون polyp کوچک است، $\alpha > \beta$ (مثلاً $\alpha = 0.7, \beta = 0.3$) کمک می‌کند مدل های polyp جا افتاده را کمتر از دست بدهد.

جدول ۳: حالت‌های خاص پارامترهای Tversky

نتیجه	β	α
Dice Loss	۰/۵	۰/۵
Jaccard / IoU Loss	۱/۰	۱/۰
جریمه‌ی بیشتر FN (مناسب پولیپ کوچک)	۰/۳	۰/۷
جریمه‌ی بیشتر FP	۰/۷	۰/۳

۴.۴ کد کامل

```

1 def tversky_loss(y_true, y_pred, alpha=0.5, beta=0.5, smooth=1.0):
2     y_true = y_true.to(dtype=y_pred.dtype)
3     tp = torch.sum(y_true * y_pred, dim=(1, 2, 3))
4     fn = torch.sum(y_true * (1.0 - y_pred), dim=(1, 2, 3))
5     fp = torch.sum((1.0 - y_true) * y_pred, dim=(1, 2, 3))
6     tversky = (tp + smooth) / (tp + alpha * fn + beta * fp + smooth)
7     return 1.0 - tversky.mean()

```

۵.۴ تحلیل تبدیل نوع

```

1 y_true = y_true.to(dtype=y_pred.dtype)

```

- هدف: تضمین یکسان بودن dtype برای جلوگیری از خطای AMP (FP16/BF16).
- در polyp معمولاً هر دو float32 هستند و این خط no-op است. ولی برای safety مفید است.

۵ بخش ۴: get_dice_ji_torch

۱.۵ هدف

محاسبه‌ی coefficient Dice و Index Jaccard به صورت per-sample بدون GPU sync.

۲.۵ مشکل در روش‌های naive

۱.۲.۵ مشکل ۱: item() سریع را می‌کشد

```

1 tp = (predict & target).sum().item() # <- GPU sync!

```

هر item() = یک GPU → CPU sync. برای ۷۹۸ نمونه $3 \times \text{sync} = 2394$.

۲.۲.۵ مشکل ۲: Aggregation سطح batch

اگر sum روی همه‌ی ابعاد (شامل batch انجام شود، Dice برای کل batch محاسبه می‌شود) → نتایج Optimistic.

۳.۵ راه‌حل: محاسبه‌ی per-sample

```

1 dims = (-3, -2, -1) # C, H, W (فقط spatial و channel)
2 tp = (predict & target).sum(dim=dims).float() # (B,)

```

۴.۵ مدیریت frames polyp-free

```
1 ji = torch.where(denom_j == 0, torch.ones_like(tp), tp / (denom_j + eps))
2 dice = torch.where(
3     denom_d == 0, torch.ones_like(tp), (2.0 * tp) / (denom_d + eps)
4 )
```

منطق:

- اگر predict و target هر دو خالی باشند ($\text{denom} = 0$) $\rightarrow$ امتیاز $1/0$ = (تشخیص درست).
- اگر فقط یکی خالی باشد $\rightarrow$ امتیاز $0/0$ =.
- در بقیه حالات $\rightarrow$ محاسبه‌ی نرمال.

۶ بخش ۵: create_experiment_folder

۱.۶ هدف

ساخت پوشه‌ی شماره‌گذاری‌شده برای ذخیره‌ی نتایج هر اجرا بدون overwrite.

۲.۶ منطق

۱. اسکن پوشه‌ی base_path برای پوشه‌های expN .

۲. یافتن بالاترین N موجود.

۳. ساخت $\text{exp}\{N+1\}$.

۳.۶ مثال

پوشه‌ی جدید	محتوای base_path
exp0	خالی
exp1	exp0
exp2	exp1, exp0
exp3	exp2, exp0 (حذف شده)

نام‌گذاری: از exp (به جای gpu) استفاده می‌شود که استاندارد آکادمیک است.

۷ بخش ۶: gen_step – گام آموزش

۱.۷ هدف

یک گام آموزش با Accumulation Gradient و Precision Mixed.


```

1 def gen_step(optimizer, gts, masks, criterion,
2               accumulation_steps=4, step=0, is_last_step=False,
3               scaler=None, tversky_alpha=0.5, tversky_beta=0.5):
4     # 1) Ensure gts is 4D and contiguous
5     if gts.ndim == 3:
6         gts = gts.unsqueeze(1)
7     gts = gts.contiguous()
8
9     # 2) Resize GT to match predicted mask size
10    size = masks.shape[2:]
11    gts_sized = F.interpolate(gts, size=size, mode="nearest")
12
13    # 3) Combined loss
14    loss = (criterion(masks, gts_sized)
15            + tversky_loss(gts_sized, masks,
16                           alpha=tversky_alpha, beta=tversky_beta))
17
18    # 4) Normalize by accumulation steps (CRITICAL)
19    loss_normalized = loss / accumulation_steps
20
21    # 5) Backward
22    if scaler is not None:
23        scaler.scale(loss_normalized).backward()
24    else:
25        loss_normalized.backward()
26
27    # 6) Optimizer update
28    should_update = ((step + 1) % accumulation_steps == 0) or is_last_step
29    if should_update:
30        if scaler is not None:
31            scaler.step(optimizer)
32            scaler.update()
33        else:
34            optimizer.step()
35            optimizer.zero_grad()
36
37    return loss.item()

```

۳.۷ نکات حیاتی

۱.۳.۷ نکته ۱: تقسیم loss بر accumulation_steps

چرا؟ اگر loss تقسیم نشود، gradient به اندازه‌ی accumulation_steps برابر بزرگ‌تر می‌شود → Rate Learning مؤثر به همان اندازه بزرگ‌تر → ناپایداری آموزش.

۲.۳.۷ نکته ۲: is_last_step – جلوگیری از نشت گرادیان

بدون این پارامتر: اگر تعداد batch در epoch مضرب accumulation_steps نباشد، gradient‌های باقی‌مانده از epoch قبلی به epoch بعدی نشت می‌کنند → آموزش غیرقابل بازتولید.

سناریو با accumulation_steps=4 و batch ۵ در epoch:

۳.۳.۷ نکته ۳: contiguous().

توضیح	should_update	is_last	step
accumulate	False	False	۰
accumulate	False	False	۱
accumulate	False	False	۲
step + zero_grad	True	False	۳
(flush) step + zero_grad	True	True	۴

```
1 gts = gts.contiguous()
```

دلیل: unsqueeze یک view ایجاد می‌کند. اگر بعداً in-place op روی tensor انجام شود، روی tensor اصلی اثر می‌گذارد. contiguous() یک copy مطمئن ایجاد می‌کند.

۸ بخش ۷: get_input_dict

۱.۸ هدف

ساخت لیست دیکشنری برای ورودی SAM (هر دیکشنری برای یک نمونه).

۲.۸ ساختار خروجی

هر دیکشنری شامل:

توضیح	شکل	کلید
تصویر	(C, H, W)	image
ابعاد اصلی	$(H_{\text{orig}}, W_{\text{orig}})$	original_size
ابعاد فعلی	$(H_{\text{curr}}, W_{\text{curr}})$	image_size
(اختیاری) نقاط	$(1, N, ۲)$	point_coords
(اختیاری) برچسب نقاط	$(1, N)$	point_labels
(اختیاری) جعبه‌ها	$(1, ۴)$	boxes
(اختیاری) ماسک‌های ورودی	$(1, 1, H, W)$	mask_inputs

۳.۸ اصلاح باگ Prompt Sharing

باگ در کد اصلی:

```
1 for i, img in enumerate(imgs):
2     single_input["boxes"] = boxes    # <- tensor! کل
```

این باعث می‌شد کل boxes به هر نمونه الصاق شود.

راه‌حل:

```
1 if boxes is not None:
2     assert boxes.shape[0] == B, "boxes batch size mismatch"
3     single_input["boxes"] = boxes[i].unsqueeze(0)    # (1, 4)
```

۴.۸ فرمت ورودی SAM

SAM انتظار دارد هر prompt یک بُعد batch محلی داشته باشد:

فیلد	شکل مورد انتظار SAM
point_coords	$(1, N, 2)$
point_labels	$(1, N)$
boxes	$(1, 4)$
mask_inputs	$(1, 1, H, W)$

۹ بخش ۸: postprocess_masks

۱.۹ هدف

جمع‌آوری خروجی‌های SAM از لیست دیکشنری‌ها به های batched tensor.

۲.۹ مدیریت Multi-mask

SAM می‌تواند ۳ ماسک برگرداند (با multimask_output=True). این تابع بهترین ماسک را بر اساس IoU prediction انتخاب می‌کند:

```
1 iou_flat = iou_pred.flatten()
2 best_idx = int(iou_flat.argmax().item())
3 cur_mask_2d = cur_mask[best_idx]
4 ious[i] = iou_value # <- IoU متناظر با همان ماسک
```

نکته: ماسک و IoU انتخاب‌شده همگون هستند. این مشکل mismatch در نسخه‌های قبلی را حل می‌کند.

۳.۹ Safety Device

```
1 if cur_mask.device != device:
2     cur_mask = cur_mask.to(device)
3 if iou_pred is not None and iou_pred.device != device:
4     iou_pred = iou_pred.to(device)
```

چرا؟ اگر masks_dict روی GPU باشد ولی device='cpu' پاس داده شود، خطای Device Mismatch رخ می‌دهد.

۱۰ بخش ۹: str2bool

۱.۱۰ هدف

تبدیل مقادیر مختلف (رشته، عدد، bool) به bool برای argparse.

۲.۱۰ مقادیر قابل قبول

نکته: مقادیر رشته‌ای case-insensitive هستند و whitespace (strip) حذف می‌شود.

ورودی	خروجی
(bool) False, True	همان مقدار
(int) 0, 1	False, True
'no', 'yes'	False, True
'false', 'true'	False, True
'f', 't'	False, True
'n', 'y'	False, True
'0', '1'	False, True

۳.۱۰ بهبود نسبت به نسخه‌ی اصلی

ویژگی	نسخه‌ی اصلی	نسخه‌ی نهایی
پشتیبانی از int	×	✓ (فقط ۰ و ۱)
whitespace strip	×	✓
پیام خطای واضح	×	✓
frozenset (سرعت)	×	✓

۱۱ بخش ۱۰: disable_batchnorm_running_stats

۱.۱۱ هدف

غیرفعال کردن running statistics در لایه‌های BatchNorm.

۲.۱۱ مشکل روش قدیمی

روش: naive

```
1 module.track_running_stats = False
2 module.running_mean = None
3 module.running_var = None
```

این کار در زمان eval خطای زیر می‌دهد:

RuntimeError: running_mean must be defined

۳.۱۱ راه‌حل مدرن: torch.func

```
1 from torch.func import replace_all_batch_norm_modules_
2 replace_all_batch_norm_modules_(model)
```

این تابع رسمی، وضعیت داخلی را سازگار نگه می‌دارد.

۴.۱۱ شمارش با _BatchNorm

```
count = sum(1 for m in model.modules() if isinstance(m, _BatchNorm))
```

مزیت: `_BatchNorm` تمام زیرکلاس‌ها را پوشش می‌دهد:

• `BatchNorm3d`، `BatchNorm2d`، `BatchNorm1d`

• `SyncBatchNorm` (مهم برای multi-GPU)

• هر زیرکلاس آینده

۵.۱۱ هشدار مهم برای polyp

مشکل بحرانی: در `polyp`، ارزیابی با `batch_size=1` انجام می‌شود. اگر `running stats` غیرفعال باشد:

• `BatchNorm` در `eval` از `batch statistics` استفاده می‌کند.

• با `batch_size=1`: `variance` → `= NaN`.

• نتیجه: ارزیابی کاملاً بی‌معنا.

راه‌حل: در `polyp` این تابع را فعال نکن.

```
args['disable_batchnorm_running_stats'] = False # پیش‌فرض
```

۱۲ نکات پایانی و توصیه‌ها

۱.۱۲ جدول تصمیم‌گیری نهایی

جدول ۴: تصمیمات برای هر تابع

#	تابع	توصیه
۱	<code>set_seed</code>	✓ استفاده با <code>strict=True</code>
۲	<code>logits_to_probs</code>	✓ استفاده (به جای <code>Min-Max</code>)
۳	<code>tversky_loss</code>	✓ استفاده (اگر لازم)
۴	<code>get_dice_ji_torch</code>	△ <code>MONAI</code> بهتر است
۵	<code>create_experiment_folder</code>	△ <code>Saver</code> بهتر است
۶	<code>gen_step</code>	△ <code>train_monai</code> استفاده می‌شود
۷	<code>get_input_dict</code>	✓ استفاده
۸	<code>postprocess_masks</code>	△ در <code>polyp</code> استفاده نمی‌شود
۹	<code>str2bool</code>	✓ استفاده در <code>argparse</code>
۱۰	<code>disable_batchnorm_running_stats</code>	× در <code>polyp</code> فعال نکن

۱. **Sigmoid** برای تبدیل logit به احتمال، علمی و صحیح است.
۲. **Accumulation Gradient** بدون تقسیم loss و بدون `is_last_step`، آموزش را ناپایدار می‌کند.
۳. **Bug Sharing Prompt** با پر-sample کردن prompt ها حل می‌شود.
۴. **Multi-mask** باید mask و IoU همگون داشته باشد.
۵. `disable_batchnorm_running_stats` در polyp فعال نکن (`batch_size=1`).